

# **RA4 - Desplegament en contenedor corporatiu**

Alex Potlog

# Index

|                                                                                   |          |
|-----------------------------------------------------------------------------------|----------|
| <b>1. L'empresa en crisi</b>                                                      | <b>3</b> |
| Quins problemes poden aparèixer si tot el codi està en un únic fitxer JavaScript? | 3        |
| Per què un projecte petit pot funcionar bé però un projecte gran es descontrola?  | 3        |
| Quin creus que és el problema principal: el llenguatge o l'organització?          | 3        |
| <b>2. Detecta l'estat</b>                                                         | <b>4</b> |
| Quins elements són "estat"?                                                       | 4        |
| Què passa si l'estat canvia però la interfície no s'actualitza?                   | 4        |
| Escriu un exemple real d'error visual que podria passar.                          | 4        |
| <b>3. Pensament crític</b>                                                        | <b>5</b> |
| Com s'actualitza la interfície quan canvia una dada?                              | 5        |
| Quin risc té aquest sistema quan l'aplicació creix?                               | 5        |
| <b>4. La revolució</b>                                                            | <b>6</b> |
| <b>5. Mini arquitectura</b>                                                       | <b>7</b> |
| Com ho dividiria?                                                                 | 7        |
| <b>6. Pregunta de reflexió</b>                                                    | <b>8</b> |
| Com organitzaria una app gran?                                                    | 8        |
| <b>7. Codi</b>                                                                    | <b>9</b> |
| On està l'estat?                                                                  | 9        |
| Què passa si hi ha 5 llocs que mostren el contador?                               | 9        |
| Què passaria si oblidem actualitzar un?                                           | 9        |

# 1. L'empresa en crisi

Quins problemes poden aparèixer si tot el codi està en un únic fitxer JavaScript?

Pot tardar molt en carregar la pàgina web, pot ser molt difícil d'editar les funcionalitats si no saps l'scope que tenen les diverses funcions. Pots acabar editant un mètode que creus que només es fa servir en una part del codi i acabar editant una funcionalitat clau i fent malbé la plataforma d'aquesta manera.

Per què un projecte petit pot funcionar bé però un projecte gran es descontrola?

En un projecte petit sol ser el cas que només es fan sol·licituts a un monolit, per exemple, o no hi ha tantes variables a tenir en compte al desenvolupar aquest apartat. En un projecte gran de més de 100k línies és usual l'ús de microserveis igual que diverses pàgines molt diferents entre elles, si això evoluciona molt acaba costant molt la correcció d'errors entre altres.

Quin creus que és el problema principal: el llenguatge o l'organització?

Trobo que principalment el problema pot sorgir d'ambdues, però pel tipatge de javascript entre altres raons trobo que és més problema de js.

## 2. Detecta l'estat

Quins elements són “estat”?

Llista d'alumnes i cercador.

Què passa si l'estat canvia però la interfície no s'actualitza?

Es produeix una desincronització entre dades i UI, que és la font principal de bugs visuals. Exemples concrets:

Elimines un alumne de la base de dades: la llista segueix mostrant-lo.

L'usuari fa logout: el nom d'usuari segueix apareixent a la capçalera.

Guardes l'edició d'un alumne: el formulari segueix mostrant les dades antigues.

Es resol un error: el missatge d'error segueix visible.

La interfície menteix a l'usuari sobre l'estat real del sistema.

Escriu un exemple real d'error visual que podria passar.

Es podria tenir una app simple de CRUD, on en eliminar un alumne no es registri i segueixi a la llista d'alumnes.

### 3. Pensament crític

Com s'actualitza la interfície quan canvia una dada?

Quan una dada canvia, el programador ha d'agafar l'element del DOM i modificar-lo a mà. Primer trobes l'element amb `getElementById`, després escoltes quan passa alguna cosa amb `addEventListener`, i finalment canvies el que es veu amb `textContent` o `innerHTML`.

Quin risc té aquest sistema quan l'aplicació creix?

El problema fonamental és que les dades viuen en variables JavaScript i la UI viu al DOM, i res els sincronitza automàticament. Ets tu qui ha de mantenir-los coherents, sempre, a mà.

El primer risc és el codi duplicat. Has d'actualitzar el DOM en cada lloc on les dades canvien: quan afegeixes un alumne, quan l'elimines, quan l'edites, quan cerques. Si t'oblides d'un sol cas, la UI queda desincronitzada i l'usuari veu dades incorrectes sense saber-ho.

El segon risc és la fragilitat dels selectors. Si canvies el nom d'un id al HTML, tots els `getElementById` que el referencien deixen de funcionar. En una aplicació gran, això és difícil de rastrejar.

El tercer risc és que no hi ha una font de veritat única. Les dades reals estan a les variables, però sovint els programadors llegeixen l'estat actual directament del DOM, és a dir, miren el que hi ha a la pantalla per saber "quina dada tenim ara". Això és un error greu, perquè el DOM és la representació visual, no el magatzem de dades.

El resultat final és una aplicació on ningú sap amb certesa quin és l'estat real del sistema. Per això van néixer frameworks com React o Vue: per invertir el model. En lloc de tu actualitzar la UI quan canvien les dades, li dius al framework quines són les dades, i ell s'encarrega de la UI automàticament.

## 4. La revolució

JavaScript tradicional no escala. Quan una aplicació supera una certa complexitat, mantenir la UI sincronitzada amb les dades a mà es converteix en un problema impossible de gestionar de forma fiable.

## 5. Mini arquitectura

Com ho dividiria?

Components per a tot el que és visual. Cada peça de la interfície és un component independent: el botó, el formulari, la taula d'alumnes, el menú de navegació. Cada desenvolupador pot treballar en els seus components sense tocar els dels altres. Si un component falla, l'error està localitzat i no enfonsa tota l'aplicació.

Mòduls per agrupar funcionalitat. Els components relacionats s'agrupen en mòduls per domini: un mòdul per a alumnes, un per a professors, un per a autenticació. Cada mòdul és responsabilitat d'un desenvolupador o parella de desenvolupadors. Ningú ha d'entendre tot el projecte per poder treballar.

Separació frontend i backend. El frontend s'ocupa de la interfície i l'experiència d'usuari. El backend s'ocupa de les dades, la lògica de negoci i la seguretat. Es comuniquen a través d'una API. Això permet que dos desenvolupadors treballin en paral·lel sense bloquejar-se: el frontend pot usar dades simulades mentre el backend no està llest.

## 6. Pregunta de reflexió

Com organitzaria una app gran?

Primer, inventaria els meus propis components. JavaScript modern permet crear Web Components natius, una funcionalitat del navegador que no necessita cap llibreria. Pots encapsular HTML, CSS i lògica en un element personalitzat reutilitzable. No és tan elegant com React, però funciona i el navegador ho entén de forma nativa.

Segon, crearia un sistema d'estat centralitzat mínim. El problema principal sense frameworks és la desincronització entre dades i UI. La solució és tenir un únic objecte que conté tot l'estat de l'aplicació, i una funció que cada vegada que l'estat canvia, recalcula i actualitza la UI. És bàsicament inventar-se un Redux mínim a mà.

Tercer, separaria el codi en mòduls amb els imports natius d'ES6. Els navegadors moderns suporten import i export sense necessitar cap eina externa. Cada fitxer té una responsabilitat clara i s'importa on es necessita.

Quart, per a la navegació entre les 10 pantalles usaria la History API del navegador, que permet canviar la URL i mostrar contingut diferent sense recarregar la pàgina. És el que fan tots els routers dels frameworks per sota.

## 7. Codi

On està l'estat?

L'estat és la variable contador. Viu en memòria com una variable global, solta, sense cap protecció ni estructura. Qualsevol part del codi pot llegir-la o modificar-la directament sense passar per cap control.

Què passa si hi ha 5 llocs que mostren el contador?

Has d'actualitzar els cinc a mà, dins la mateixa funció:

```
function sumar() {  
  contador++;  
  document.getElementById("valor-1").textContent = contador;  
  document.getElementById("valor-2").textContent = contador;  
  document.getElementById("valor-3").textContent = contador;  
  document.getElementById("valor-4").textContent = contador;  
  document.getElementById("valor-5").textContent = contador;  
}
```

El codi ja comença a fer mal als ulls. I encara no hem afegit cap funcionalitat real.

Què passaria si oblidem actualitzar un?

Aquí està el problema de fons. L'aplicació continua funcionant sense donar cap error. El contador real val, posem, 8. Però un dels elements de la pantalla segueix mostrant 6. No hi ha cap avís, cap excepció, cap senyal que alguna cosa va malament.

L'usuari veu dades contradictòries a la mateixa pantalla i no sap a quina creure. El desenvolupador no sap on és el problema perquè el codi no es queixa. I com més gran és l'aplicació, més difícil és trobar quin element s'ha quedat desactualitzat.

Aquest és exactament el problema que frameworks com React resolen: en lloc de dir-li al navegador "actualitza aquest element concret", li dius "l'estat val 8" i ell s'encarrega que tots els llocs que mostren l'estat reflecteixin el valor correcte, sempre, automàticament.